

Deploying an Nginx Web Server on AWS EC2 Inside a Custom VPC

Lab Assignment 3 — AWS, VPC and Instance Deployment

IT-457 Cloud Computing — Dhirubhai Ambani University

Group: 202611022 — 202611047 — 202611030

Aim. To understand the fundamentals of Amazon Web Services (AWS), provision a custom Virtual Private Cloud (VPC) with its associated networking resources, and launch an EC2 instance that installs and starts an Nginx web server automatically via a user-data script, so that its status can be verified both from a browser and from the instance itself. The work was done in the `ap-south-1` (Asia Pacific, Mumbai) region under AWS account 789655958513.

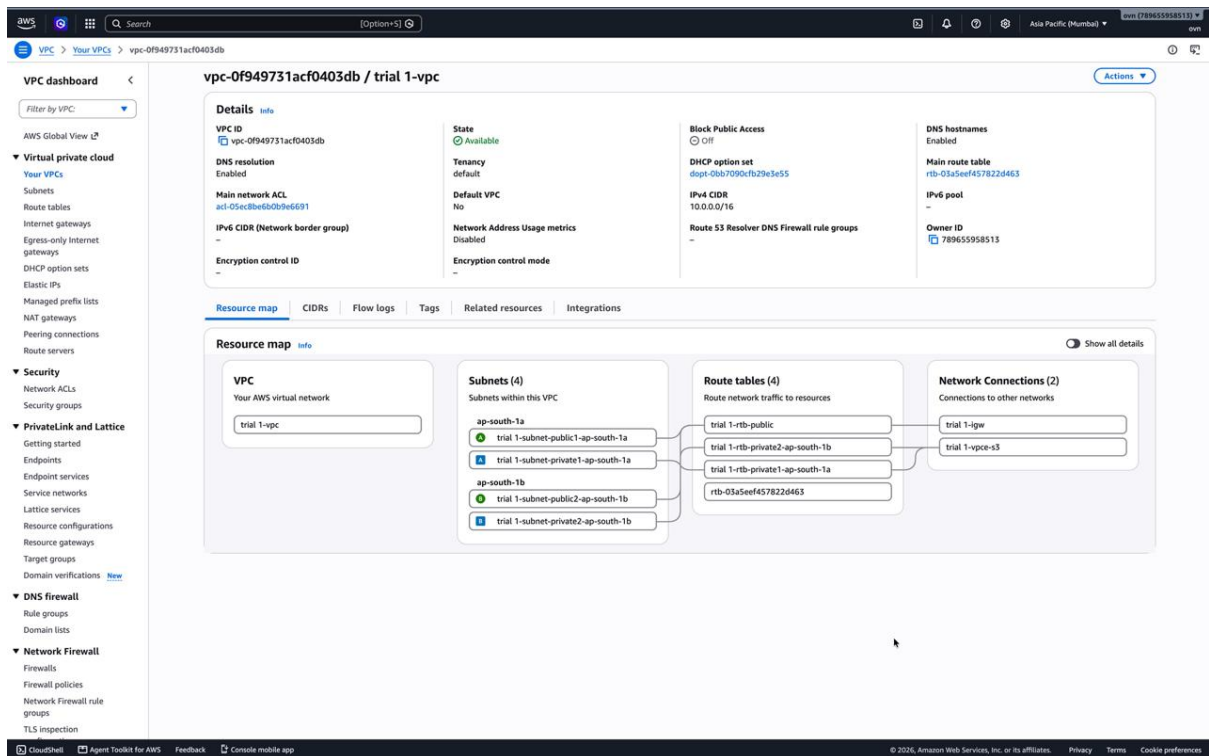
1. Basics of AWS

AWS is Amazon’s public-cloud platform: instead of buying and maintaining physical servers, we rent compute, storage and networking resources on demand and pay only for what is used. The lab introduced a few core building blocks:

- **Region and Availability Zone (AZ).** A *region* is an independent geographic area (we used `ap-south-1`, Mumbai). Each region is split into multiple *AZs* — isolated data centres such as `ap-south-1a` and `ap-south-1b` — so resources can be spread out for resilience.
- **EC2 (Elastic Compute Cloud).** On-demand virtual machines, called *instances*. We used a `t3.micro` instance running Ubuntu 24.04 LTS.
- **VPC (Virtual Private Cloud).** An isolated virtual network inside a region that we fully control: our own IP address range, subnets, route tables and firewalls.
- **Security Groups.** A stateful, instance-level firewall that decides which inbound and outbound traffic is permitted.
- **User data.** A shell script supplied at launch time that AWS executes once, automatically, the first time the instance boots — used here to install and configure Nginx without any manual SSH login.

2. Creating the VPC

Rather than launching into the account’s default VPC, we created a dedicated VPC (shown in the console as `trial 1-vpc`) using the “VPC and more” wizard, which creates the VPC together with its subnets, route tables, internet gateway and an S3 endpoint in a single step. We used the IPv4 CIDR block `10.0.0.0/16`, which reserves 65,536 private addresses that AWS then divides across the subnets.

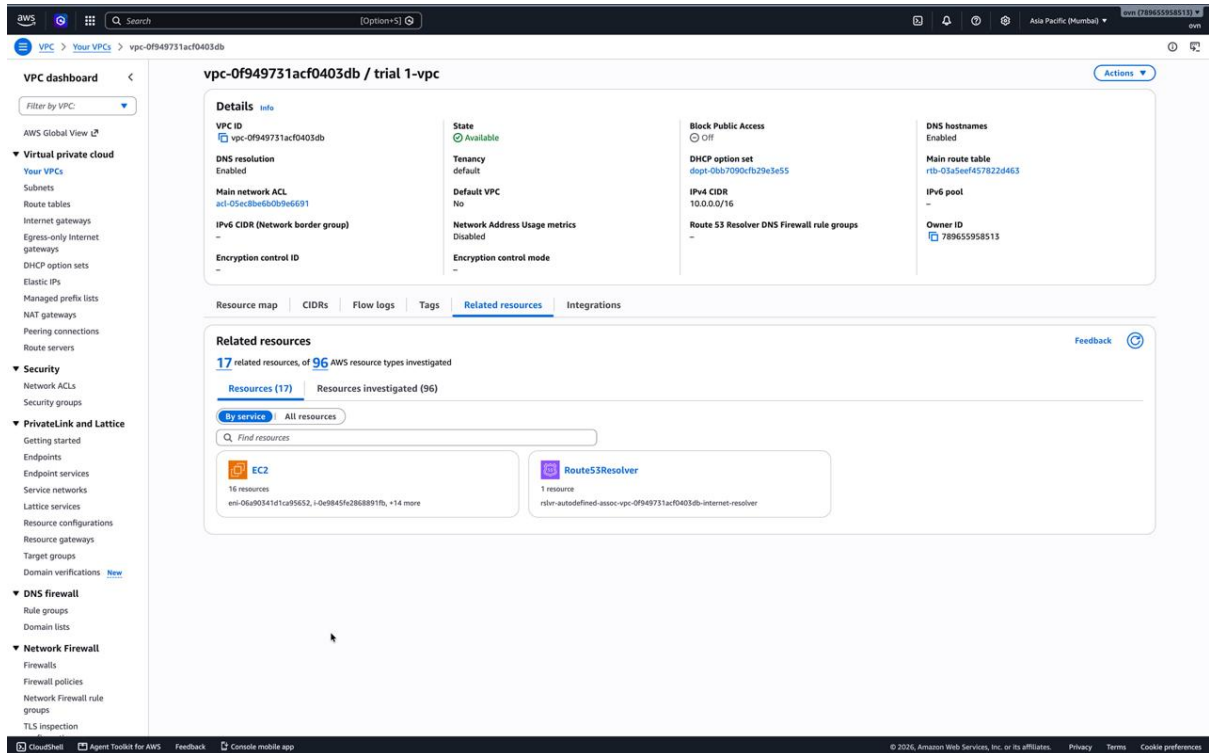


The resource map above is a useful summary of everything the wizard provisioned:

Property	Value
VPC ID	vpc-0f949731acf0403db (trial 1-vpc)
State	Available
IPv4 CIDR	10.0.0.0/16
Tenancy	default
DNS resolution	Enabled
DNS hostnames	Enabled
Main route table	rtb-03a5eef457822d463
Main network ACL	acl-05ec8be6b0b9e6691
DHCP option set	dopt-0bb7090cfb29e3e55

- **Subnets (4).** One public and one private subnet in each of two AZs: `trial 1-subnet-public1-ap-south-1a` / `trial 1-subnet-private1-ap-south-1a` in `ap-south-1a`, and a matching public/private pair in `ap-south-1b`. Public subnets can be reached from the internet; private subnets cannot.
- **Route tables (4).** `trial 1-rtb-public` routes `0.0.0.0/0` traffic to the internet gateway; the private route tables (`trial 1-rtb-private1-ap-south-1a`, `trial 1-rtb-private2-ap-south-1b`) keep their subnets' traffic off the public internet.
- **Internet Gateway.** `trial 1-igw` is what actually connects the VPC to the internet and is the target of the public route table's default route.
- **S3 Gateway Endpoint.** `trial 1-vpce-s3` allows instances to reach Amazon S3 over AWS's internal network instead of going out through the internet gateway.

The **Related resources** view confirms that the VPC is not just an empty container: it already has 17 dependent resources, including our EC2 instance and its network interface, and a Route 53 resolver used for DNS.



3. Network Settings

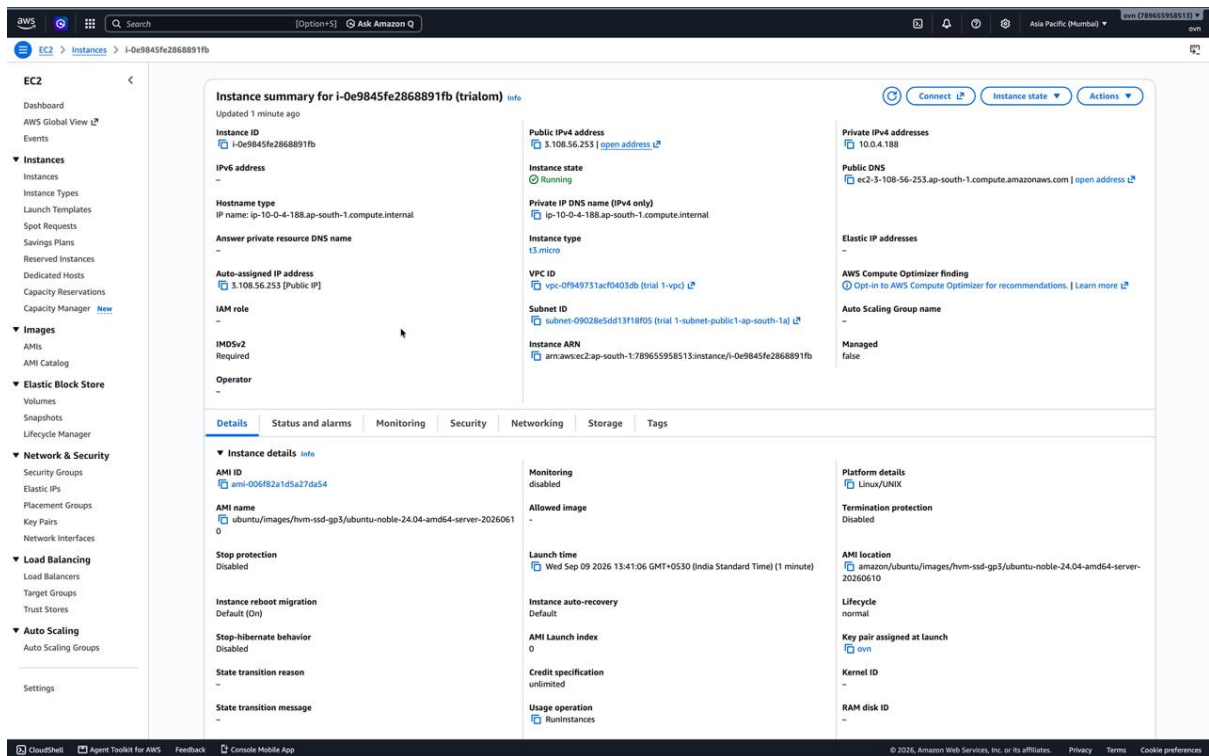
To make the instance reachable over HTTP while keeping it otherwise locked down, three network decisions were made at launch time:

- **Subnet placement.** The instance was launched into the *public* subnet `trial 1-subnet-public1-ap-south-1a` in AZ `ap-south-1a`, so it sits on the side of the network that has a route to the internet gateway.
- **Auto-assign public IP.** Enabled for this launch, which is why the instance was given a public IPv4 address in addition to its private VPC address.
- **Security Group.** Configured to allow inbound HTTP (TCP 80) from `0.0.0.0/0` so the Nginx page is publicly reachable. No inbound SSH rule was added, so the instance is deliberately *HTTP-accessible only* — it cannot be reached by SSH from outside; any further interaction with the box (e.g. the shutdown in Section 5) was done through the browser-based EC2 Instance Connect session instead of a key-based SSH login.

This combination — public subnet, a route to the internet gateway, an auto-assigned public IP, and a security-group rule opening port 80 — is exactly what is needed for the web page to load in Section 5, while every other port stays closed by default.

4. Launching the Instance

A `t3.micro` instance running Ubuntu 24.04 LTS (`ami-006f82a1d5a27da54`) was launched into the public subnet described above, with `IMDSv2` set to *required* so metadata requests must use the token-based flow rather than the older, less secure `v1` calls. The instance's own summary page reports the following configuration once it was running:



Property	Value
Instance ID	i-0e9845fe2868891fb
Instance type	t3.micro
Availability Zone	ap-south-1a
Private IPv4	10.0.4.188
Public IPv4	3.108.56.253
Public DNS	ec2-3-108-56-253.ap-south-1.compute.amazonaws.com
Subnet	trial 1-subnet-public1-ap-south-1a
AMI	Ubuntu 24.04, ami-006f82a1d5a27da54
IMDSv2	Required

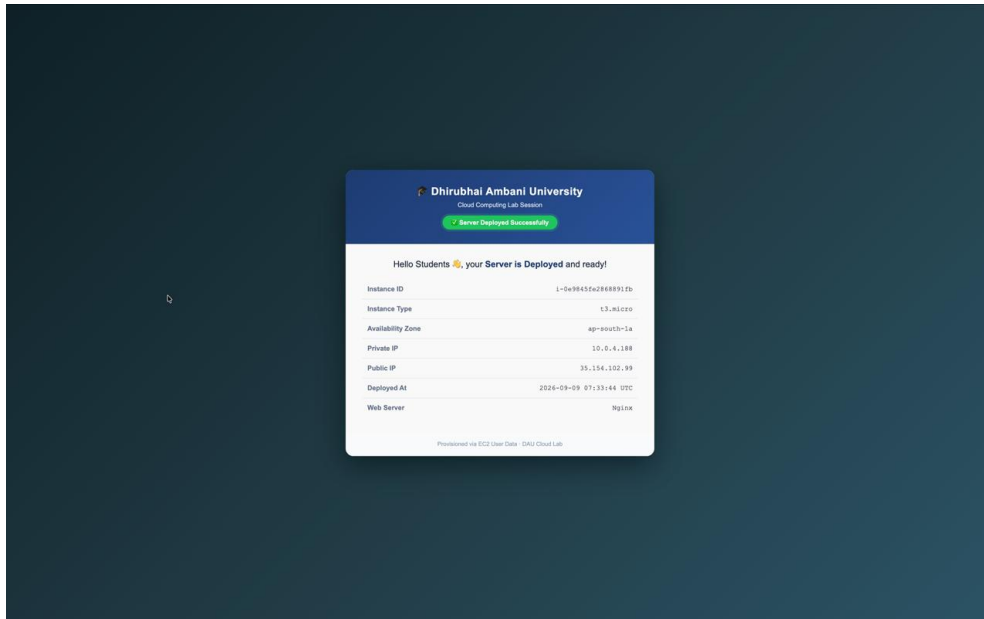
A user-data script was pasted into the launch wizard so that Nginx would be installed and started automatically, with no manual login required. At a high level the script:

1. installs the **nginx** package with **apt-get**;
2. requests an IMDSv2 token and uses it to query the instance's own metadata (instance ID, instance type, AZ, private/public IP) from the link-local metadata endpoint **169.254.169.254**;
3. writes those values into an HTML page saved to **/var/www/html/index.html**, which Nginx serves by default; and
4. enables and (re)starts the **nginx** service so the page is live immediately and the service also comes back up automatically after any future reboot.

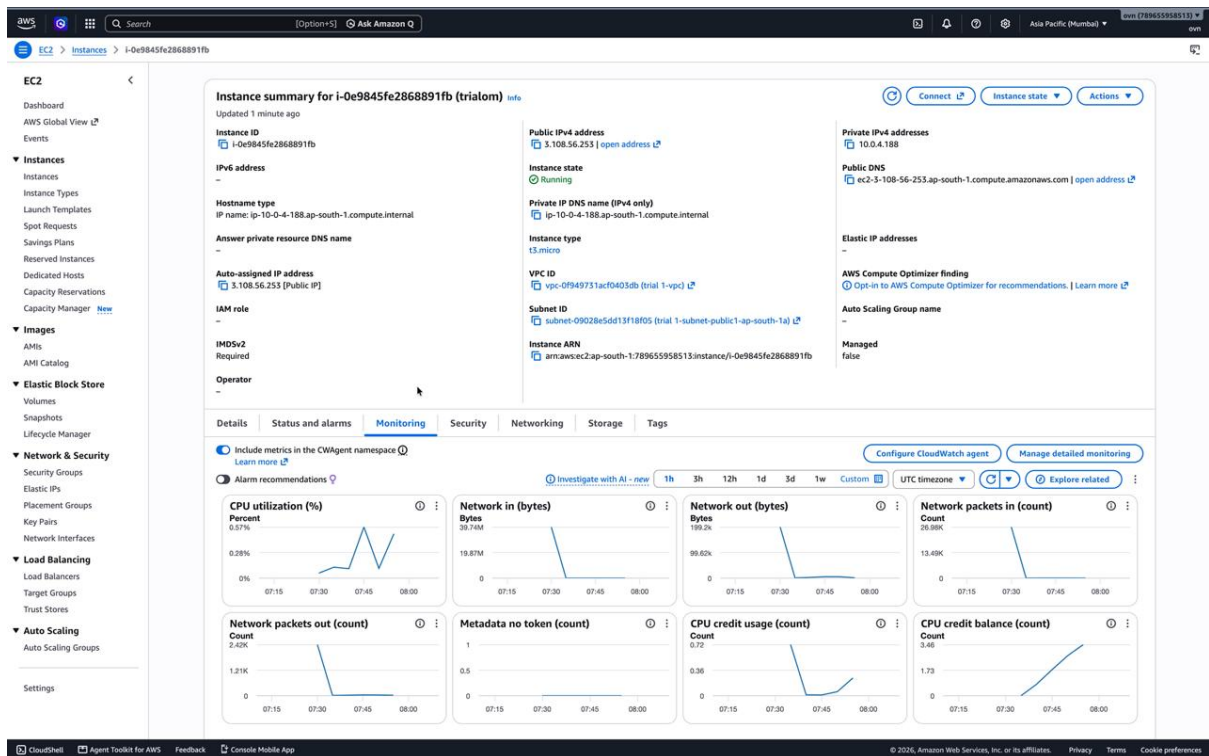
Because the script reads the instance's own metadata rather than hard-coding any values, the same script works unmodified no matter which instance or IP it happens to run on.

5. Verification

(a) **The instance is publicly reachable over HTTP.** Browsing to the instance's public address returns the page generated by the user-data script, confirming that the subnet, routing, public IP and security-group rule from Section 3 all worked together as intended. The metadata printed on the page matches the instance shown in Section 4 (instance ID `i-0e9845fe2868891fb`, type `t3.micro`, AZ `ap-south-1a`, private IP `10.0.4.188`), deployed at 2026-09-09 07:33:44 UTC; the public IP shown here differs slightly from the earlier console screenshot because AWS re-assigns a fresh public address whenever the instance is stopped and started again.



(b) **Live monitoring confirms the instance was actually running the workload.** The CloudWatch metrics on the instance's Monitoring tab show a burst of CPU utilisation and network activity around the time of launch, consistent with the boot process downloading and installing the Nginx package before settling into an idle baseline once the page was up:



(c) **Cleaning up.** Once the deployment was verified, the instance was connected to via the browser-based EC2 Instance Connect session (not SSH, since no SSH rule was ever opened) and shut down with a normal power-off, to avoid leaving billable compute running after the lab was complete:

```

Welcome to Ubuntu 24.04.4 LTS (GNU/Linux 6.17.0-1017-aws x86_64)
 * Documentation: https://help.ubuntu.com
 * Management: https://landscape.canonical.com
 * Support: https://ubuntu.com/support

System information as of Wed Sep 9 07:47:11 UTC 2026
System load: 0.0          Temperature: -273.1 C
Usage of /: 32.0% of 10.3GB Processes: 111
Memory usage: 27%        Users logged in: 0
Swap usage: 0%           IPv4 address for ena1: 10.0.4.188

Expanded Security Maintenance for Applications is not enabled.
148 updates can be applied immediately.
127 of these updates are standard security updates.
To see these additional updates run: apt list --upgradable
Enable ESM Apps to receive additional future security updates.
See https://ubuntu.com/esm or run: sudo pro status

The programs included with the Ubuntu system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.
Ubuntu comes with ABSOLUTELY NO WARRANTY, to the extent permitted by
applicable law.

To run a command as administrator (user "root"), use "sudo <command>".
See "man sudo_root" for details.

ubuntu@ip-10-0-4-188:~$
Broadcast message from root@ip-10-0-4-188 (Wed 2026-09-09 07:51:08 UTC):
The system will power off now!

```

Observations

- Reachability over HTTP was never a property of the EC2 instance alone — it depended on the whole chain of VPC decisions: public subnet, a route to the internet gateway, an auto-assigned public IP, and a security group that explicitly opens port 80.
- Leaving SSH closed and relying on EC2 Instance Connect for the one administrative task we

needed (shutting the box down) shows that a security group can be scoped tightly to only the access a deployment actually requires.

- Driving the whole setup from user data meant the Nginx installation, metadata lookup and page generation happened automatically on first boot, with nothing typed by hand on the instance itself.

Conclusion

- Building a dedicated VPC (10.0.0.0/16) with its own public/private subnets, route tables, internet gateway and S3 endpoint gives full control over an instance's network environment, instead of relying on the shared default VPC.
- Whether a server can be reached from the internet is fundamentally a networking question — subnet placement, routing, a public IP and the right security-group rule all have to line up, not just the server software running on the instance.
- EC2 user data turns the first-boot configuration of a server into a repeatable script: one file installed Nginx, pulled the instance's own metadata through IMDSv2, and published a status page automatically, with no manual setup step.